VibeHub

Türkiye'nin sanatçı odaklı merch pazaryeri

Platform Architecture

Engineering reference for the VibeHub platform

Audience · Engineering team, technical advisors
Date · 2026-05-28
Source · docs/ARCHITECTURE.md

vibehub.com.tr · api.vibehub.com.tr

Contents

1. Overview

VibeHub is a Turkish multi-vendor artist merchandise marketplace. Customers ("fans") browse merch from music artists, comedians, influencers, and other creators ("vendors"), pay via Iyzico (placeholder today), receive shipments tracked through Turkish carriers (Aras / Yurtiçi), and can request refunds through a return-shipment workflow with VH-RET-XXXXXXXX barcodes.

- **Production:** https://vibehub.com.tr (frontend) / https://api.vibehub.com.tr (API)
- **Languages:** Turkish (default) + English, in-code i18n via `lib/i18n.ts`
- **Deployment:** Single VPS, Docker Compose, auto-deploy on push to `main`

2. Stack & Tooling

Layer	Technology
Backend	NestJS 10 + Prisma 5 + PostgreSQL 16 + Redis 7 (BullMQ)
Frontend	Next.js 14 App Router + Tailwind + Zustand + TanStack Query
Mobile	Expo SDK 52, React Native 0.76, expo-router v4, NativeWind
Auth	JWT (access 15m, refresh 90d) + MFA via emailed OTP + trusted devices (365d)
Search	Meilisearch (optional — falls back to Prisma LIKE)
Mail	Nodemailer + SMTP, BullMQ-queued
E-invoice	Foriba (mock fallback)
Payment	Iyzico (mock fallback)
Shipping	Aras / Yurtiçi (mock fallback)
Push	Expo Push (mock fallback)
Error reporting	Sentry (backend + mobile)
Container	Docker Compose (one VPS, ~4 services)
CI/CD	GitHub Actions (deploy.yml on push to main + ci.yml gating + backup-verify.yml daily)

3. Monorepo Layout

```
/Users/emregercek/Desktop/VibeHub/
├── backend/                  # NestJS API (own package.json, own npm workspace)
│   ├── src/
│   │   ├── admin/            # /admin/* routes — god-class controller
│   │   ├── auth/             # JWT + OTP + trusted devices
│   │   ├── order/            # Order lifecycle + refund workflow
│   │   ├── kargo/            # Shipment + return barcode generation
│   │   ├── payment/          # Iyzico + mock pay
│   │   ├── permissions/      # 15 vendor permissions enum
│   │   ├── mail/             # 9 email templates
│   │   ├── queue/            # BullMQ processors
│   │   ├── scheduler/        # Cron jobs (security digest)
│   │   ├── search/           # Meilisearch + fallback
│   │   ├── audit/            # Audit log writes
│   │   ├── ...               # ~31 modules total
│   │   └── main.ts
│   ├── prisma/
│   │   ├── schema.prisma     # 36 models, 19 enums
│   │   ├── migrations/       # 35 migrations — sacred history
│   │   └── seed.ts           # Creates GOD_USER from env
│   ├── Dockerfile
│   ├── package.json
│   └── .env                  # Local-only secrets (gitignored)
│
├── frontend/                 # Next.js 14 App Router (own package.json)
│   ├── app/
│   │   ├── (public)/         # Marketing, legal, support, vendor-apply
│   │   ├── auth/             # Login + verify + reset
│   │   ├── shop/             # Product listing
│   │   ├── product/[id]/     # PDP
│   │   ├── store/[slug]/     # Vendor storefront
│   │   ├── profile/          # Customer account
│   │   ├── dashboard/admin/  # Admin panel (~20 pages)
│   │   ├── dashboard/vendor/ # Vendor panel
│   │   └── ...
│   ├── components/
│   ├── hooks/
│   ├── lib/                  # api.ts, i18n.ts, format.ts, etc.
│   ├── store/                # Zustand (auth, theme, toast)
│   ├── public/
│   ├── Dockerfile
│   └── package.json
│
├── mobile/                   # Expo app — SEPARATE git repo (EmreGerck/vibehub-mobile)
│   ├── app/                  # expo-router screens
│   │   ├── (auth)/
│   │   ├── (tabs)/           # home / shop / forum / profile / scan
│   │   ├── cart.tsx          # Stack screen (NOT a tab)
│   │   └── _layout.tsx
│   ├── src/
│   │   ├── api/              # Axios wrappers
│   │   ├── components/       # SplashOverlay + shared UI
│   │   ├── theme/            # tokens.ts + useTheme.ts (system-default)
│   │   ├── store/            # authStore (zustand + secure-store)
```

```
│   │   └── utils/
│   ├── assets/
│   └── package.json
│
├── .github/workflows/
│   ├── ci.yml                # Type-check on PR
│   ├── deploy.yml            # SSH → VPS → docker compose up --build
│   └── backup-verify.yml     # Daily 03:00 UTC — verifies DB backups exist
│
├── docs/
│   ├── ARCHITECTURE.md       # This file
│   └── INFRASTRUCTURE.md     # LOCAL ONLY (gitignored) — credentials + VPS details
│
├── docker-compose.yml        # Production stack
├── docker-compose.uat.yml    # UAT stack (separate DB, ports 4000/4001)
├── package.json              # npm workspaces config (backend + frontend; mobile excluded)
├── Dockerfile                # Root-level (used by something?)
└── README.md
```

Workspaces: Root `package.json` declares `["backend", "frontend"]`. Mobile is intentionally separate (different toolchain).

---

4. Deployment Topology

Production VPS

Single VPS (`vibehub.com.tr`), Nginx → 4 Docker containers, all bound to 127.0.0.1:

Container	Internal Port	Purpose
`vibehub_postgres`	5432	PostgreSQL 16, volume `postgres_data`
`vibehub_redis`	6379	Redis 7 (BullMQ + OTP storage), volume `redis_data`
`vibehub_backend`	3001	NestJS API, volume `backend_uploads`
`vibehub_frontend`	3000	Next.js production server

Nginx terminates SSL and routes:

- `vibehub.com.tr` → `localhost:3000`
- `api.vibehub.com.tr` → `localhost:3001`

UAT stack (`docker-compose.uat.yml`) runs on the same VPS, separate DB + Redis index, ports 4000/4001 (`uat.vibehub.com.tr`).

CI/CD (GitHub Actions)

Push to `main` triggers `.github/workflows/deploy.yml`:

1. `ci-gate` **job** (Node 20):
   - `npm ci` for all workspaces
   - `prisma generate` in backend
   - `npx tsc --noEmit` for backend + frontend
2. `deploy-backend` **job** (SSH via `appleboy/ssh-action`):
   - `cd /root/vibehub && git fetch && git reset --hard origin/main`
   - Prune docker build cache older than 72h (prevents disk-full)
   - `docker compose up -d --build backend`
   - Health check loop: `curl /app-config` every 2s, max 40s
3. `deploy-frontend` **job**:
   - Re-build with `NEXT_PUBLIC_API_URL` + `NEXT_PUBLIC_SITE_URL` baked in
   - `docker compose up -d --build frontend`

Migrations run automatically on backend container startup via `prisma migrate deploy`.

Backup verification (`backup-verify.yml`) runs daily at 03:00 UTC.

Local Development

```
# From repo root
docker-compose up -d                # Postgres + Redis only
cd backend && npm run start:dev     # API on :3001
cd frontend && npm run dev          # Web on :3000

# Mobile (separate repo)
cd mobile && npx expo start --port 8083
```

---

5. Database

Schema

`backend/prisma/schema.prisma` — **36 models, 19 enums, 35 migrations**.

Key Models

Identity & Access:

- `User` (role enum: CUSTOMER, VENDOR_OWNER, VENDOR_MANAGER, PLATFORM_ADMIN, GOD_USER)
- `RefreshToken`, `TrustedDevice`, `PasswordResetToken`

- `UserProfile`, `ProfileVisit`, `DirectMessage`

Multi-tenancy:

- `Tenant` (vendors) — status: PENDING / ACTIVE / FROZEN / REJECTED
- `TenantPermission` (per-vendor permission grant)

Commerce:

- `Product`, `ProductVariant`, `Category`
- `Order`, `OrderItem`
- `Shipment`, `ReturnShipment`
- `Payout`
- `Cart` (server-side, authoritative)
- `Wishlist`, `Review`, `Follow`

Content & Marketing:

- `HeroBanner`
- `ForumSettings`, `ForumChannel`, `ForumTopic`, `ForumReply`, `ForumReaction`
- `VendorMedia` (Spotify/YouTube embeds), `VendorEvent`
- `NfcTag`

Platform:

- `PlatformSettings`, `AppConfig`, `AppNotification`
- `PushDevice`
- `AuditLog`
- `PageView` (device analytics)

Key Enums

```
UserRole          CUSTOMER | VENDOR_OWNER | VENDOR_MANAGER | PLATFORM_ADMIN | GOD_USER
TenantStatus      PENDING | ACTIVE | FROZEN | REJECTED                    ⚠ admin UI bu
ProductStatus     DRAFT | PENDING_REVIEW | LIVE | ARCHIVED
OrderStatus       PLACED | CONFIRMED | SHIPPED | DELIVERED | CANCELLED | REFUND_REQUESTED
ShipmentStatus    CREATED | IN_TRANSIT | DELIVERED | FAILED
ReturnShipmentStatus  INITIATED | DROPPED_OFF | IN_TRANSIT | ARRIVED_AT_DEPOT | COMPLETED
PayoutStatus      PENDING | PROCESSING | PAID | FAILED
PreOrderStatus    (used in product pre-order state machine)
NotificationType  (in-app notification kinds)
DevicePlatform    IOS | ANDROID | WEB
```

Migration Discipline

Migrations under `prisma/migrations/` are **sacred history** — never reorder or rewrite. The most recent five:

1. `20260526220000_return_shipment`

2. `20260528000000_order_lifecycle_timestamps`
3. `20260528010000_sprint8_money_fixes`
4. `20260528020000_nfc_industrial`
5. `20260528030000_page_view_analytics`

---

6. Backend

Module Map (31 modules under `backend/src/`)

Module	Route Prefix	Purpose
`app`	—	Root + global guards (`JwtAuthGuard` , `RolesGuard`)
`auth`	`/auth`	Register, login, OTP, MFA, refresh, password reset, devices
`admin`	`/admin`	All admin operations — **god-class controller (1700+ lines)**
`vendor`	`/vendors`	Vendor apply, list, store profile
`product`	`/products`	Public catalog + vendor CRUD
`category`	(implicit)	Category tree CRUD
`cart`	`/cart`	Server-authoritative cart
`order`	`/orders`	Customer + vendor + admin order operations + refund workflow
`payment`	`/payments`	Iyzico + mock pay
`kargo`	`/kargo`	Shipment create + tracking + return barcode
`payout`	`/payouts`	Vendor payouts (request + admin approve)
`review`	`/reviews`	Product reviews
`wishlist`	`/wishlist`	Customer wishlist
`profile` (userprofile)	`/user-profile`	Public + private user profiles
`messages`	`/messages`	Direct messages between users
`notifications`	`/notifications`	In-app notifications
`push`	—	Expo Push integration
`devices`	`/devices`	Push device registration
`mail`	—	Mail templates + queue producer
`queue`	—	BullMQ module + processors
`scheduler`	—	Cron jobs (daily security digest)
`forum`	`/forum`	Channels, topics, replies, reactions
`media`	`/media`	Vendor Spotify/YouTube embeds
`event`	—	Vendor event listings
`nfc`	`/nfc`	NFC tag CRUD + scan redirect + bulk industrial tools
`banner`	`/banners`	Hero banners

Module	Route Prefix	Purpose
`search`	`/search`	Meilisearch + fallback
`merchant-feed`	(implicit)	RSS 2.0 product feed for Google Shopping
`einvoice`	`/einvoice`	Foriba e-invoice (mock fallback)
`permissions`	—	Vendor permission CRUD service
`audit`	—	Audit log writes
`redis`	—	Global Redis client provider
`prisma`	—	Global Prisma client provider
`common`	—	Decorators, guards, response DTOs
`trap`	(implicit)	Honeypot for prompt-injection detection
`sso`	—	Reserved for future SSO
`app-config`	`/app-config`	Public bootstrap config
`analytics`	`/analytics`	Vendor + admin analytics
`storage` (upload)	`/upload`	File upload to local disk

~256 endpoints total across all controllers.

Authentication

- **Access token**: JWT, 15-minute TTL, `JWT_ACCESS_SECRET`
- **Refresh token**: 90 days, stored in `RefreshToken` table, rotated on use
- **Trusted device**: 365 days, stored in `TrustedDevice` table; if present + valid, MFA OTP is skipped
- **MFA**: 6-digit OTP, 5-minute TTL, max 5 wrong guesses, 30s resend cooldown, 5/min IP rate limit
- **Account lockout**: exponential — 1m → 5m → 15m → 1h after 5 failed logins in 15-minute window
- **Mobile bypass**: `/auth/login` (legacy endpoint) does NOT require OTP — used by mobile until trusted-device handshake completes
- **Web flow**: `/auth/login/mfa` is mandatory — always returns challenge, frontend prompts for OTP unless device is trusted

Authorization

Two layers:

1. **Role-based** via `@Roles(UserRole.X)` decorator + `RolesGuard` (global APP_GUARD). PLATFORM_ADMIN treated as ADMIN; GOD_USER inherits all admin privileges plus 4 exclusive ops (create admins, reindex, send security digest, view all DMs).
2. **Permission-based** for vendors via `VendorPermission` enum + `PermissionsService.assert()`. 15 distinct permissions:
   - In `DEFAULT_VENDOR_PERMISSIONS`: PRODUCT_CREATE, PRODUCT_EDIT, PRODUCT_DELETE, PRODUCT_SUBMIT, VARIANT_MANAGE, INVENTORY_EDIT, ORDER_VIEW, ORDER_FULFILL, STOREFRONT_EDIT, PAYOUT_REQUEST, ANALYTICS_VIEW, MANAGER_INVITE, FORUM_MANAGE
   - **Opt-in only** (not in defaults): PRODUCT_PUBLISH_DIRECT, MEDIA_MANAGE

Background Jobs (BullMQ)

Queue	Processor	Job Types
`mail`	`queue/processors/mail.processor.ts`	sendOtp, sendVendorWelcome, sendPasswordReset, sendOrderConfirmation, sendOrderToVendor, sendShipmentUpdate, sendRefundApproval, sendReturnShipmentLabel, sendSecurityAlert

Cron jobs via `@nestjs/schedule`:

- `scheduler/security-digest.service.ts` — daily at 08:00 Istanbul (`SECURITY_DIGEST_CRON=0 5 * * *` UTC). Sends digest of critical/warning audit events to all PLATFORM_ADMIN + GOD_USER.

Email Templates (9)

All in `mail/mail.service.ts`. Dark-themed HTML, dark navy background, gradient CTAs.

Template	Triggered By	Recipient
`sendOtp`	Login / register	User
`sendVendorWelcome`	Admin approves vendor	Vendor owner
`sendPasswordReset`	Forgot password	User
`sendOrderConfirmation`	Payment confirmed	Customer
`sendOrderToVendor`	Order placed	Vendor
`sendShipmentUpdate`	Order → SHIPPED	Customer
`sendRefundApproval`	Admin approves refund	Customer
`sendReturnShipmentLabel`	Refund initiated	Customer
`sendSecurityAlert`	Daily cron	All admins

Audit Log

59 distinct event types, written via `audit.service.log()`. Severity tiers:

- **Critical**: `ACCOUNT_LOCKED`, `ADMIN_ORDER_STATUS_OVERRIDE`, `ADMIN_USER_PASSWORD_RESET`, `PLATFORM_SETTINGS_UPDATE`, `TRAP_ROUTE_HIT`
- **Warning**: Most operational actions (vendor / product / order management)
- **Info**: `LOGIN_SUCCESS`, `PASSWORD_CHANGED`

⚠ **Known coverage gaps:** banner CRUD, push broadcast, depot arrival are NOT audit-logged in some paths.

Throttling

Via `@nestjs/throttler`:

- Login / register: **10/min** per IP
- OTP verify / send: **5/min** per IP
- Forgot password: **5/hour** per IP
- Password reset email (resend): **3/hour** per IP

7. Frontend

Route Tree

```
/                                  Home
/about, /contact, /privacy, /terms, /kvkk, /legal, /rehber, /support
/vendors/apply

/auth
  /login                           Email + password → MFA challenge
  /verify                          OTP entry
  /register
  /forgot-password
  /reset-password

/shop                              Product listing
/shop/[category]                   Pre-rendered (SSG) per category
/shop/tag/[tag]                    SSR

/product/[id]                      PDP (SSR + JSON-LD)
/store/[slug]                      Vendor storefront (SSG-ish)

/u/[nickname]                      Public user profile
/cart                              Server cart view
/checkout                          Address + payment selection
/payment                           Payment result polling
/order-confirmation/[id]           Post-purchase
/invoice/[orderId]                 e-Arşiv PDF download
/nfc/[tagId]                       Server 302 to tag.destinationUrl

/profile                           Account home
  /orders, /orders/[id]            Customer orders + tracking + return barcode
  /messages, /messages/[userId]
  /notifications, /wishlist
  /visitors, /social
  /password, /settings

/dashboard/admin (20 pages)        See ADMIN section below
/dashboard/vendor (~9 pages)       See VENDOR section below
```

State Management

- **Zustand stores** (`store/`):
  - `auth.store.ts` — user, accessToken, refresh state. Subscribed across all protected pages.
  - `theme.store.ts` — dark-mode toggle.
  - `toast.store.ts` — global toast notifications.
  - ~~`cart.store.ts`~~ — **REMOVED 2026-05-28** (server cart is authoritative).

- **React Query** for server state — `lib/api.ts` axios singleton has a refresh-token race-guard for concurrent 401 retries.

Component Patterns

- **Page structure**: Server Component `page.tsx` (with `generateMetadata`) + separate `*Client.tsx` for interactivity.
- **CSS classes**: `.card`, `.btn-primary`, `.btn-ghost`, `.input`, `.badge-*` defined in `globals.css`.
- **i18n**: `const t = useI18n((s) => s.t)` → `t('namespace.key')`. TR keys ~line 28, EN ~line 1100 in `lib/i18n.ts`.
- **Dark mode**: always pair `text-gray-900 dark:text-white`, `bg-white dark:bg-black`.
- **Animations**: `animate-fade-in-up`, `animate-scale-in`, `<Reveal>` wrapper.
- **SEO**: `components/seo/JsonLd.tsx` + `ProductFaqJsonLd.tsx` for structured data; `sitemap.ts`, `robots.ts`, `manifest.json` in `app/`.

---

8. Mobile (Expo)

Separate git repo: `EmreGerck/vibehub-mobile`. Sits in monorepo at `mobile/` for convenience.

Route Tree (expo-router v4)

```
app/
├── _layout.tsx                  Root provider (QueryClient, SafeArea, Splash)
├── index.tsx                    Initial entry → routes based on auth state
├── (auth)/_layout.tsx           Auth stack — login, register, forgot
├── (tabs)/_layout.tsx           Bottom tabs
│   ├── home/                    Feed
│   ├── shop/                    Product list + sort/filter
│   ├── forum/                   Forum browse
│   ├── profile/                 Account + settings + orders + wishlist
│   └── scan/                    NFC tag scanner (camera + nfc-manager)
├── cart.tsx                     Stack screen (NOT a tab)
├── checkout.tsx                 Stack screen
├── product/[id].tsx
├── vendor/[slug].tsx
└── order/[id].tsx
```

Session Strategy

- **Tokens stored** via `src/utils/storage.ts` → `expo-secure-store` on native, `localStorage` on web fallback
- `authStore` hydrates on `_layout.tsx` mount → silent `POST /auth/refresh-mobile` if refresh token exists

- **90-day refresh + 365-day trusted device** = user effectively never re-logs-in unless explicit logout

Splash

`src/components/SplashOverlay.tsx` — cinematic diagonal-split reveal:

- Backdrop fades in
- "Vibe" spring-slams from left (purple → pink gradient)
- "Hub" spring-slams from right (pink → amber gradient)
- Crack line ignites between them
- NW/SE halves slide apart along diagonal
- App revealed underneath

Min display time = 2800ms (`MIN_SPLASH_MS` in `app/_layout.tsx`) — doubles as auth hydrate buffer.

Theme

`src/theme/tokens.ts` + `useTheme.ts` → `useColorScheme()` system-default with `Palette.brand = #9333EA` matching desktop.

Native Modules

`expo-secure-store` , `expo-notifications` , `expo-camera` , `expo-barcode-scanner` , `react-native-nfc-manager` , `expo-local-authentication` , `expo-image-picker` , `expo-calendar` .

---

9. Third-Party Integrations

Service	Purpose	Env Vars (keys)	State
Iyzico	Card payment	`IYZICO_API_KEY`, `IYZICO_SECRET_KEY`, `IYZICO_BASE_URL`, `IYZICO_CALLBACK_URL`	Mock fallback active — no real keys
Foriba	Turkish e-Arşiv invoice	`FORIBA_API_KEY`, `FORIBA_BASE_URL`	Mock fallback active
Aras Kargo	Shipping label + tracking	(none yet — placeholder)	`_mockCreate` / `_mockTrack` returns DEMO data
Yurtiçi Kargo	Shipping label + tracking	(none yet — placeholder)	`_mockCreate` / `_mockTrack` returns DEMO data
SMTP (Resend / generic)	Email	`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	Live — falls back to stdout logging if unset
Sentry	Error monitoring	`SENTRY_DSN` (backend + mobile)	Live
Meilisearch	Full-text product search	`MEILISEARCH_HOST`, `MEILISEARCH_API_KEY`	Optional — falls back to Prisma LIKE
Expo Push	Mobile push notifications	(Expo handles automatically)	Live (token registration works, no campaign UI)
Neon Postgres	DB hosting (local dev option)	`DATABASE_URL`	Used for local dev — VPS uses self-hosted Docker Postgres
Upstash Redis	Optional Redis SaaS	`REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `REDIS_TLS=true`	Available — VPS uses self-hosted Docker Redis
Google Merchant Center	Product feed	(output only)	Live — RSS 2.0 at `/merchant-feed` endpoint

Hardcoded TC kimlik (`'11111111111'`) in `payment.controller.ts` for invoice issuing — known placeholder.

10. Business Rules & Constants

Auth Lifetimes

- Access token: **15m** (`JWT_ACCESS_EXPIRES_IN`)
- Refresh token: **90d** (`REFRESH_TOKEN_LIFETIME_DAYS = 90` in `auth.service.ts`)
- Trusted device: **365d** (`TRUSTED_DEVICE_LIFETIME_DAYS = 365`)
- OTP code: **6 digits**, **5min TTL**, **5 max attempts**, **30s resend cooldown**

Return Shipment

- Barcode format: `VH-RET-` + 8 uppercase hex characters → e.g. `VH-RET-A1B2C3D4`

Throttling

- Login / register / OTP verify: **10/min** per IP
- OTP resend: **5/min** per IP (+30s in-memory cooldown)
- Forgot password: **5/hour** per IP
- Password reset email resend: **3/hour** per IP

Account Lockout (exponential)

- 5 failed logins / 15min window → 1st lockout: **1m**
- 2nd: **5m**, 3rd: **15m**, 4th+: **1h**

Money

- Currency: **TRY** (fixed)
- Commission rate: per-tenant `commissionRate` field, default set via `PlatformSettings.defaultCommissionRate`
- Free shipping threshold: stored in `PlatformSettings.freeShippingThreshold` — **decision pending**

Limits

- Max upload file size: **10 MB** (`MAX_FILE_SIZE_MB`)
- Storage: local disk `backend/uploads/` (S3-ready architecture, not migrated yet)

11. Workflows

Customer (Fan)

1. **Register** `/auth/register` → auto-login (no email verification step)

2. **Login** `/auth/login` → MFA challenge → `/auth/verify` for OTP → optional device-trust
3. **Browse** `/` → `/shop` → `/product/[id]` → `/store/[slug]`
4. **Cart** `POST /cart/items` → `/cart`
5. **Checkout** `/checkout` (collects shipping address) → `POST /orders`
6. **Pay** `/payment?orderId=` → `POST /payments/mock/pay` (real Iyzico stubbed)
7. **Confirmation** `/order-confirmation/[id]` — auto-issues e-Arşiv invoice
8. **Track** `/profile/orders/[id]` → polls `GET /kargo/track/:trackingNumber` every 5m
9. **Refund** `PATCH /orders/my/:id/request-refund` → generates `VH-RET-XXXXXXXX` barcode → email with carrier drop-off instructions
10. **NFC Scan** `/nfc/[tagId]` → server 302 redirect

Vendor

1. **Apply** `/vendors/apply` → `POST /vendors/apply` (creates `Tenant{PENDING}` + `User{VENDOR_OWNER}` + default permissions). ⚠ **No email to admin or applicant.**
2. **Wait for approval** (admin polls `/dashboard/admin/vendors`)
3. **Approved** → `VENDOR_WELCOME` email
4. **Storefront** `/dashboard/vendor/profile` → `PATCH /vendors/me` (logo/banner URL only, no upload UI)
5. **Add product** `/dashboard/vendor/products` → `POST /products` (title/price/description only — **no images, variants, or stock UI**)
6. **Submit for review** `PATCH /products/:id/submit` (direct-publish only with opt-in permission)
7. **Receive order** — ⚠ no notification at all
8. **"Fulfill"** Mark Confirmed → Mark Shipped → `PATCH /orders/vendor/:id/status` → customer email goes with `trackingNumber: null` (no vendor-side shipment creation UI)
9. **Payout** `/dashboard/vendor/payouts` — ⚠ **page is broken** (calls admin endpoint)

Admin

Sidebar has 20 items in 6 groups (`components/admin/AdminSidebar.tsx`):

- **Genel Bakış** — Dashboard, Analitik (GOD-only)
- **Satıcı Yönetimi** — Satıcılar, Ürün Onayları, Kategoriler, Yorumlar, Medya
- **Siparis & Finans** — Siparişler, Ön Siparişler, Ödemeler, Finansal
- **İçerik & Pazarlama** — İndirimler, SEO Motoru, Hero Bannerlar, Etkinlikler, NFC Etiketleri, Mobil Uygulama
- **Kullanıcılar & Güvenlik** — Kullanıcılar, Güvenlik Monitörü, İşlem Kaydı
- **Platform Ayarları** — Platform Ayarları

Key flows:

- **Approve vendor**: `PATCH /admin/vendors/:id/status` — triggers welcome email; reject **never emails reason**
- **Approve refund**: ✅ new `PATCH /order/admin/:id/approve-refund` (notifies + return-shipment integration) — ⚠ legacy `PATCH /admin/orders/:id/refund` button still wired (skips notification)
- **Confirm depot arrival**: `PATCH /kargo/return/:orderId/arrived` → status `ARRIVED_AT_DEPOT`. ⚠ Not audit-logged, no connection to refund approval
- **Push broadcast**: `POST /admin/notifications/push-broadcast` — single button, no audience targeting, no audit log

---

12. Known Issues (Prioritized) — Sprint 12 Status

✅ P0 Security — RESOLVED in Sprint 12

1. ~~`POST /payments/iyzico/refund/:paymentId` no `@Roles`~~ → **verified guarded** (`payment.controller.ts:130`)
2. ~~`POST /payments/iyzico/verify/:token` no `@Roles`~~ → **verified guarded** (`payment.controller.ts:120`)
3. ~~`POST /payments/mock/pay` not env-gated~~ → **verified gated** (`payment.controller.ts:148-150` throws `NotFoundException` in production)
4. `PATCH /admin/settings` — swagger says GOD-only, class-level guard requires PLATFORM_ADMIN — still open; class-level guard is the actual enforcement; swagger is misleading. **Trivial fix:** add `@Roles(UserRole.GOD_USER)` to the method.

✅ P0 Data / Workflow — MOSTLY RESOLVED in Sprint 12

1. ~~TenantStatus PENDING_REVIEW enum mismatch~~ → **verified fixed** (admin UI uses `'PENDING'` consistently)
2. ~~Vendor cannot edit products after create~~ → **NEW** comprehensive editor at `/dashboard/vendor/products/[id]`
3. ~~Vendor cannot add variants or stock via UI~~ → **NEW** variant table + stock adjustment in editor
4. ~~Vendor cannot upload images~~ → **NEW** multipart upload via `POST /upload/image`
5. ~~Vendor payouts page → 403~~ → **FIXED** — switched to `GET /payouts/mine`, added `POST /payouts/request` for self-service
6. ~~No vendor-side shipment creation~~ → **NEW** modal with carrier + tracking; `POST /kargo/shipments` runs before status transition
7. ~~Two duplicate refund paths in admin UI~~ → **REMOVED** legacy `PATCH /admin/orders/:id/refund` route + service method + hook + DTO + UI

✅ P1 Notifications — RESOLVED in Sprint 12

- **Vendor new-order email** → `sendVendorNewOrder` template + per-tenant grouping in `order.service.placeOrder`
- **Vendor refund-request email** → `sendVendorRefundRequest` template + trigger in `order.service.requestRefund`
- **Depot arrival notification** → already in place (`kargo.service.confirmDepotArrival` notifies + audit-logs)
- Still open: no admin email on vendor apply, no applicant confirmation, no reject notice with reason — these are vendor-application workflow gaps not addressed in Sprint 12

✅ P1 Confirmations — RESOLVED in Sprint 12

Vendor Freeze/Approve/Reject, Product Approve, Push broadcast, Maintenance mode, Banner delete, Product delete — all one-click without confirmation step.

🟡 P1 — Permission Inconsistencies

- `PRODUCT_PUBLISH_DIRECT` + `MEDIA_MANAGE` opt-in but UI doesn't gate
- `MANAGER_INVITE` permission exists but has no backend route — dead
- Frontend `useCan()` only used on `/dashboard/vendor/products`
- Analytics: frontend GOD-gated, backend permits PLATFORM_ADMIN — bypassable

🟢 P2 — i18n / Code Quality

- Hardcoded Turkish on i18n-aware pages: `/payment/*`, `/order-confirmation/*`, `/profile/orders/*`
- Hardcoded English on Turkish-default site: `/u/[nickname]`, `/profile/visitors`, `/profile/messages`
- `useI18n.getState()` called in render → locale switch doesn't update strings
- `admin.service.ts` is 1764 lines — god-class
- `<tbody>` nested in `<tbody>` in admin orders page

---

13. Sprint 12 Additions Summary (2026-05-28)

New endpoints

- `POST /payouts/request` — vendor self-requests payout (auto-period, auto-amounts)
- `GET /admin/queue-health` — BullMQ mail queue depth (waiting/active/failed/completed)
- `GET /admin/business-metrics` — GMV / activation / refund rate / cart abandonment / time-to-first-order

New backend services

- `scheduler/audit-retention.service.ts` — monthly cron prunes AuditLog rows > 12 months old
- `admin/business-metrics.service.ts` — Prisma aggregates with 60s in-memory cache

New migrations

- `20260528040000_add_hot_path_indexes` — composite indexes on Order, Product, OrderItem, AppNotification (uses `CREATE INDEX CONCURRENTLY` — runs outside transaction safely on prod)

New mail templates

- `sendVendorNewOrder(to, orderId, storeName, items, currency)` — one per affected vendor on placement
- `sendVendorRefundRequest(to, orderId, storeName, customerName, reason)` — informational, all affected vendors

New frontend pages

- `app/dashboard/vendor/products/[id]/page.tsx` — comprehensive vendor product editor
- `app/faq/page.tsx` — 14 Turkish FAQs + FAQPage + BreadcrumbList JSON-LD
- `app/u/[nickname]/layout.tsx` — ProfilePage + Person JSON-LD, honors ghostMode

New frontend components

- `components/shared/PermissionDenied.tsx` — friendly empty state for missing vendor permissions
- `components/seo/HreflangTags.tsx` — Next 14 alternates helper (TR + EN + x-default)

New hooks

- `useUpdateProduct`, `useArchiveProduct`, `useUpdateVariant`, `useDeleteVariant`, `useAdjustStock`, `useUploadImage` in `hooks/useProducts.ts`
- `usePayoutsMine` in `hooks/usePayouts.ts`

Auth security hardening

- `auth.service.deleteAccount()` — full KVKK-compliant transaction (was a 1-line stub that would have crashed at runtime). Anonymizes Orders/Reviews/Forum content, purges sessions/messages/devices, retains AuditLog with `actorId=null` for forensics.

Email queue resilience

- `queue.service.ts` — mail jobs now retry 3× with exponential backoff (5s base)
- `mail.processor.ts` — `@OnWorkerEvent('failed')` listener; final failures captured to Sentry

Type safety

- New `AuthenticatedUser` interface in `common/current-user.decorator.ts`
- 13 controllers updated: `user: any` → `user: AuthenticatedUser` (~33 sites)

Tests

- New `src/__money_tests__/money-flow.spec.ts` — 3 smoke tests (order place, refund approve, payout create)
- Fixed `order.service.spec.ts` — was failing 8/8 before sprint, now 8/8 pass

Documentation

- New `docs/RUNBOOKS.md` — 8 operator procedures (backup drill, KVKK requests, soft-launch, GSC submission, deliverability, incident response, vendor smoke test, pre-deploy checklist)
- `~/.claude/skills/vibehub-workflow/SKILL.md` enriched with env catalog, VPS topology, integrations, dev commands

---

14. Recent Cleanup (2026-05-28)

Files Removed

- `frontend/store/cart.store.ts` — server cart authoritative
- `mobile/src/hooks/useAppConfig.ts` — 0 callers
- `backend/fly.toml` — stale (deploys to VPS via GH Actions, not Fly)
- `backend/dump.rdb` — local-redis side effect (now gitignored)
- `docs/mobile-preview.html`, `docs/vibehub-preview.html`, `docs/.Rhistory` — initial-commit junk

NPM Dependencies Removed

Backend:

- `@sentry/nestjs` (using `@sentry/node` directly)
- `resend` (using `nodemailer` instead)
- `@nestjs/schematics` (CLI-only, not runtime)
- `jest-mock-extended` (unused — Jest built-ins suffice)
- `supertest` + `@types/supertest` (no integration tests use it)

Frontend:

- `meilisearch` (only a string-literal reference, no real usage)

`ts-node` was kept (used by `seed` npm script).

Verification: backend + frontend pass `tsc --noEmit` and full build (`nest build` / `next build`); mobile passes `tsc --noEmit` .

14. Quick Command Reference

```
# Local dev
docker-compose up -d                          # Postgres + Redis
cd backend && npm run start:dev               # API on :3001
cd frontend && npm run dev                    # Web on :3000
cd mobile && npx expo start --port 8083       # Mobile on :8083

# Type-check / build (each workspace)
cd backend && npx tsc --noEmit && npm run build
cd frontend && npx tsc --noEmit && npm run build
cd mobile && npx tsc --noEmit

# Deploy (automatic on push to main)
git push origin main

# Database
cd backend
npx prisma migrate dev --name <name>          # local — creates new migration
npx prisma migrate deploy                      # production — applies pending
npx prisma db seed                             # creates GOD_USER from env
npx prisma studio                              # browser UI on :5555

# Seed local admin (verify-only — quick way to get GOD_USER)
# Email: <GOD_USER_EMAIL from .env>  Password: <GOD_USER_PASSWORD from .env>
# Default if env unset: god@vibehub.com.tr / God@VibeHub2025!

# Local Redis (if .env points to cloud)
brew install redis
/opt/homebrew/opt/redis/bin/redis-server --port 6379 --daemonize yes
# Then override for backend boot:
REDIS_HOST=127.0.0.1 REDIS_PORT=6379 REDIS_TLS=false REDIS_PASSWORD= node dist/main.js
```

15. Heuristics for Working in This Codebase

- **Multiple entry points** — permissions and settings have multiple admin UI pages editing the same data. Always grep before adding a new entry.
- **Vendor-facing changes must add `useCan()` gating** — only `/dashboard/vendor/products` does it correctly today.
- **Order status transitions must emit a notification** — check `notifications.create()` is called alongside any `order.update()` .

- **Mock placeholders are intentional** — don't replace them with real API calls unless explicitly told a provider has been chosen.
- **All emails go through BullMQ** via `queue/processors/mail.processor.ts` — adding a new email type requires a new `mailType` enum branch.
- **Audit logging is via** `this.audit.log()` — coverage is inconsistent; banner CRUD, push broadcast, depot arrival are gaps.
- **Migrations are immutable** — never edit a committed migration; always create a new one.
- **DTOs at controller boundary are strict** — `whitelist:true` + `forbidNonWhitelisted:true` in `main.ts`. Unknown fields → 400.